Capstone Proposal: Wayfarer’s Crawl

Wayfarer’s Crawl - Enemy AI Dungeon Crawler Video Game

By: [redacted]

[redacted]

09/09/2025

Overview

Practices and standards within the game development industry have evolved over the past few decades to better meet the demands of players and showcase advancements in the computer science field. One such way has been through the evolution of non-player character (NPC) behaviors in games, better known as game AI; this was first seen in the 1990s with popular games such as *Blade Runner* in 1997, which modeled its enemy AI with behaviors, and *Half-Life* in 1998, that had systems to place to allow for enemies to use tactics against the player. Another game that revolutionized enemy AI in gaming was *Halo 2, released* in 2004, which was the first commercially successful game to utilize behavioral trees[1] as a driver for enemy AI. Currently, these previously innovative methods are commonly deemed as industry standard due to being well researched and proven to work time and time again (Yannakakis). Many modern games, such as *The Last of Us*, use these methods to great success by utilizing data-driven programming, a development approach where game content and behavior are defined in external data files rather than hard-coded into the program's logic. This separation enables the rapid implementation of states, behaviors, and enemies. One of the main enemy types within the game was implemented within the last five months of development due to the developers staying true to the design principles behind data-driven programming (Botta).

Within this project, we aim to design enemy AI for a single-player, top-down, 2D game. The game will have the player traverse through multiple dungeons, encountering enemies along the way that use different algorithms for their logic. The enemy AI will be driven by the use of either a finite state machine[2] or a behavioral tree to determine the most appropriate action for the enemy to use at any given moment. During the first semester, the main goal will be to get

[1] you can add new states and behaviors to your character without changing the other states' logic; you just have to add new transitions.A mathematical model of plan execution that switches between a finite set of tasks in a modular fashion Ai (FSM, Behavior Tree, Goap, Utility Ai)
[2] A mathematical model of computation that can change from one state to another in response to inputs. Ai (FSM, Behavior Tree, Goap, Utility Ai)

the game into a working state with simple enemies that will react to basic player actions, whereas the second semester will be focused on implementing enemies with complex logic and providing the player with more tools that can directly manipulate the enemy AI's logic. To properly take on this project, along with the challenges we encounter throughout the creation of the enemy AI models, we are looking to develop an interface that will allow for quick transitions from one enemy state to another. This interface will primarily be designed for development and quality assurance testing.

Problem Statement

The primary goal of this capstone is to create a fun game where advanced AI makes the NPCs feel truly life-like. We want to provide team members with an in-depth understanding of behavioral trees and finite state machines by creating intelligent enemy AI that dynamically interacts with both the player and the environment. Between the player's choices and intricate enemy tactics, we want believable, intention-driven behavior, which creates the illusion of intelligence from our structured finite state machines and behavioral trees. To create these seemingly intelligent interactions without sacrificing performance, efficiently managing CPU and memory resources will be paramount to ensuring a seamless and enjoyable player experience.

Problem Solutions

Enemy AI and Design

Since this capstone is focused on enemy AI, there will need to be multiple goals for enemies and a variety of enemies to fully showcase the enemy logic. Each enemy will have a unique set of goals that it prioritizes, as well as varying complexity in how it achieves these

goals. For example, a Ranged Enemy's goals may be to keep some distance from the player and itself, while continually shooting arrows from a distance. A Melee Enemy, in contrast, will prioritize closing the distance and attacking the player, putting on pressure. This allows each enemy type to feel distinct. To achieve this, enemies will be separated into two different categories. These are basic enemies and boss enemies. Basic enemies will use a finite state machine as the driver for their logic and will be scattered throughout the dungeon. These enemies are common and, for the first semester, will be found in two different categories: Ranged and Melee. Each basic enemy will have a set of actions that can be used to achieve their main goal, regardless of their category (Ranged/Melee). The specific number of actions is undecided and will be decided based on what feels appropriate for the enemy type. This is because certain enemies may employ multiple actions to achieve their goals. This 'set' of actions could be in the form of a list containing the available actions that check the current state of the enemy; another option is for the current state of the enemy to reference one or more actions to then be executed. For example, an enemy in a "Wandering" State may only have access to the 'Walking' action, and an enemy in the "Chasing" State may only have access to attacking actions. With these two starting enemies, the groundwork will be laid for future, more complex enemies.

The Melee Enemy's primary role is to aggressively close distance and maintain constant pressure on the player. We will design Melee Enemies to be encountered in groups; they are strong enough to withstand three to four light attacks from the player, forcing the player to focus on crowds rather than on singular targets. As their primary role is to be aggressive, their behavior is straightforward: they will pathfind directly towards the player at all times. Their simple, relentless nature makes them the simplest enemy to implement. Their design creates a foundational challenge that we can later build upon with more complex behaviors.

The Ranged Enemy's primary role is to provide a long-range threat that forces the player to close the distance. Unlike the melee enemy, the Ranged Enemy's behaviour is to actively

create and maintain distance from the player. This kiting (A game design term, meaning the NPC will actively retreat to create distance while still firing projectiles) behavior ensures the enemy fulfills its designated role and gives the player a far more challenging interaction than if the enemy simply backed away. To balance their range advantage, ranged enemies are more fragile than melee enemies and can be defeated in just two to three light attacks. Their key intelligence lies in situational decision-making. They won't always retreat blindly, but based on factors like the player's health, available space, or proximity to obstacles, they might choose to hold their ground and continue attacking the player. This also prevents frustrating, endless chase sequences and creates a more dynamic and intelligent combat. A great example of this is Minecraft Dungeons Skeletons; these enemies will shoot from a distance and, when approached, will back away until at a safe distance again.

Due to the scope of work for the first semester, we will focus on the core dynamics of the AI behavior. However, there are several extra features (mentioned below) that would significantly enhance the immersion and illusion of intelligence. We will consider these features for implementation next semester. Potential second-semester additions include:

Combination Attacks: These types of attacks will be used in tandem with another attack to showcase a higher level of complexity of the enemy AI. For instance, a basic ranged enemy could use a skill to knock the player back, daze them, and then immediately launch a powerful attack while the player is still dazed. Another example could be a basic melee enemy using a skill to close the distance between them and the player, then using a skill to stun the player.

Modifiers: modifiers are 'parent' states, where each type of modifier will have a subclass. For example, an "Enraged" State, which will alter all decisions made by the NPC, to be centered around area of effect (AOE) attacks. This gives the illusion of an enraged enemy that is no longer striking meticulously at the player. This allows variation within enemy interactions.

There are additional enemy types that may be considered for next semester to provide a variety of challenges to the player, while also being able to showcase our AI. An additional two enemies that we discussed include:

- Tank: A basic enemy that is slower than the ranged/melee types, but can withstand around five to seven attacks. This enemy is meant to provide a challenge when paired with large numbers of ranged/melee enemies. Their goal would be to provide as much pressure to the player, preventing the player from engaging with other enemies.
- Healer: A more complex enemy that provides support to other enemies. This enemy has the least amount of health, typically withstanding around one to three light attacks. This enemy's goal is to heal its allies and prevent any deaths. This enemy will attempt to position itself behind any allies when possible.

We will have created one Boss enemy by the end of this semester. Boss enemies will be designed so that the player will typically need to fight them multiple times to learn the strategy and attack patterns of the boss. Due to this, the boss will need to be able to survive for a long period, longer than a basic enemy typically would, have complex moves for the player to learn, and deal enough damage to seem threatening but not impossible to defeat. Some other potential characteristics of bosses are mechanics and phases. Mechanics will move beyond simple direct attacks. The core difference between these mechanics and attacks is that a mechanic will have several stages to it. An example of a mechanic could be a boss with a hammer hitting the ground around the player. Then the ground would erupt again in a larger radius a few seconds later, damaging the player a second time if the player did not move away.

Boss phases are a core mechanic designed to maintain an engaging and challenging difficulty curve. When a boss's health is reduced to a specific threshold (We will decide what that threshold is during implementation and testing), it enters a new phase. This transition typically introduces new, more complex attack patterns and movement behaviors, refreshing the

fight and testing the player's skills further. Bosses may even have more than one phase, changing their move sets several times during one fight. These phases could potentially use different behavioral trees or just unlock new actions based on each phase; we will decide which of these two we prefer to use during implementation, based on what provides a more enjoyable gameplay loop, while not affecting performance. Since bosses have no sustainability abilities and should feel considerably different from basic enemies, they have large health pools. Bosses also have actions similar to basic enemies, but instead of a finite state machine, they will be implemented with a behavioral tree. The difference between a behavioral tree and a finite state machine will be made clear by how the boss may utilize a 'random selector'. A random selector is a parent node (A node that has one or more child nodes) that can randomize a behavioral tree's decision between its child nodes. This provides a sense of randomness to the boss that will keep players engaged. The details between the finite state machine and behavioral tree implementations will be expanded upon in the later section 'States and Behaviors'. A boss's main goal can differ depending on "what feels natural", such as if the boss mainly uses ranged attacks, it may want to avoid contact with the player. This means the aforementioned boss can teleport around the boss arena. This provides the challenge of keeping this avoidance to a manageable amount for the player to handle. A melee boss may have gap-closing abilities and prioritize applying pressure upon the player.

For the first boss enemy, we are designing the core mechanic around a boss in the game *Final Fantasy 14*. In this fight, the boss charges up an attack that increases the size of a circular attack in the center of the arena. In this video, at the time timestamp provided, this mechanic can be seen, where the boss waves their fist three times, which correlates to the attack being as large as the third ring on the floor of the arena (Mrhappy1227). The first boss in our game will use this same concept and will have access to a circular attack with scaling radius. The radius, or range of the attack, will be indicated to the player through a sprite element located above the boss's model. This sprite element could be presented as three empty

diamonds, and throughout the fight, they will begin to glow. The amount of glowing diamonds will determine the range of the circular attack, but the attack range will not be telegraphed as it is in the provided video. We are choosing this mechanic to focus on for the first boss due to it being a simple concept for the player to understand, alongside us having the sprite assets for the scaling circular attack mechanic.

States and Behaviors

To properly implement actions and skills for the basic and boss enemies within the game, they will need a logic model to realistically respond to player inputs and environmental changes. In many real-time action games, the goal of the enemies is for them to respond quickly to the player while also seeming believable in what they do. This means a simple decision-making model can be used as long as it is structured to prevent the enemies from acting in a way that wouldn't be believable.

> "Characters give the illusion of intelligence when they are placed in well-thought-out setups, are responsive to the player, play convincing animations and sounds, and behave in interesting ways. Yet all of this is easily undermined when they mindlessly run into walls or do any of the endless variety of things that plague AI characters (Botta)."

One solution to implementing this decision-making model is to incorporate simple state-based models for the enemy logic into our game. These models will be realized with finite state machines and behavioral trees, using a variety of states (called behaviors for behavioral trees) that dictate the current available actions of the enemy. In addition to state-based models, each enemy will have some ability to observe their environment or also known as "partial observability." This will allow them to take into account the current state of the game world before they transition from state to state. These types of enemies are called "model-based reflex agents", meaning they keep track of how the game world (which includes the player, other enemies, and the environment) changes over time, alongside how their actions will affect the

game world (Russell 50). Implementing an enemy that tracks the state of the game world and incorporates it into its decision-making process would be a large challenge for the first semester. Thankfully, neither finite state machines nor behavioral trees need to use the state of the game world or how it will evolve in order to function, but we would like to add that layer of complexity in the future to make the enemy AI seem more believable.

For the basic enemies within the game, we plan on using only finite state machines to control their logic. Many of the states will be shared across basic enemies such as a “Wandering” State which will be used to control what the enemies do when there is no outside factors in play, in other words, this will be the initial state and also the fallback state: The initial and fallback states act as an emergency state to go back to if something goes wrong. Table 1 below shows several other states we will be implementing. The “Wandering”, “Modifier”, and “Alerted” States are ones associated with basic enemy AI behavior, that being the enemies going from an idle animation, to being alerted by the player, to then chasing and attacking the player. The “ChaseTarget” State for following a target will be implemented for future use if enemies that follow one another are implemented. The “FollowTarget”, “DamageOverTime”, and “CrowdControl” States will be implemented and tested last, and for the most part, are states that the player can force the enemies to enter. An example of this would be if the player had access to a skill that “stuns” the enemy: this stunning skill would take parts of the “CrowdControl” State, inheriting its core functionalities, while adding new functionalities as well, and once the player uses said skill, it will send the enemy AI into a “Stun” State, which will alter what action they will take next, alongside their current behavior. Another example of a forced entry state is the “DamageOverTime” State, in that the player will have the ability to send the enemies into that state. The last state that needs to be mentioned is the “Modifier” State, however, this state we plan to implement later in the second semester, along with some child states that inherit from it, such as an “Enrage” State or “Fear” State, that will alter the enemy’s available attacks, pathfinding logic, movement speed, and field of view.

State:	Functionality:
Wandering	An initial and fallback state where the enemy moves around the map freely without a set goal
Modifier	A parent state for when the enemy is affected by a modifier status from a player or enemy skill
Alerted	The enemy has been alerted to the player either by the player or a player skill entering the enemy's field of view
ChaseTarget	The enemy is chasing a target, intending to attack them, this target likely being the player
FollowTarget	The enemy is following a target, likely another enemy
DamageOverTime	A parent state for when the enemy is inflicted with a damage over time status from an attack or skill
CrowdControl	A parent state for when the enemy is affected by a crowd control status from an attack or skill
Table 1: Basic Enemy States	

Boss enemies within the game will use behavioral trees as the primary driver for their logic. Behavioral trees are similar in concept to finite state machines, with them going from behavior to behavior instead of state to state. However, they differ greatly in how they are structured, with behavioral trees being more akin to a tree data structure rather than a finite state machine, which is more akin to a state diagram. Each node can be categorized as follows:

- Composite: Parent node with one or more child nodes. The type of composite node determines how each child node is executed and whether one or all child nodes need to return true.
- Conditional: Binary success or failure nodes that check some condition with either the game world, enemy, or player.
- Decorator: These are wrapper nodes with a single child that alter the behavior of the child node.

- Action: Leaf nodes in the behavioral tree that execute certain actions, such as animators and functions.

Each type of node has multiple variants; we plan on implementing each of the four primary node types, as well as at least one variant of each node type for each boss enemy. Some of the important variants include:

- Selector: A Composite node that returns true once one of the child nodes returns true.
- Sequence: A Composite node that returns true only if every child node returns true.
- Parallel: Composite node that executes each child node until one returns false.
- Random Selector: Selector node that randomizes the order of the child nodes.
- Execute Action: Action node that calls a function.
- Wait Action: Action node that waits a specified amount of time.

As for the first behavioral tree to be implemented, it will be used by the first boss in the game, and a rough outline of it can be seen in Figure 1. The tree, as seen in Figure 1, would cover the behaviors of the boss during the first phase of the fight. To signal to the boss enemy that a phase transition is occurring, the behavior tree has a conditional as the first child node from the root. After this is a random selector to choose between two boss mechanics. These boss mechanics are a sequence of actions (the second boss mechanic being minimized in the figure to save space) that will be attempted one at a time. If any action fails within the mechanic sequences, the behavioral tree will go to the normal attack sequence, where the boss will locate the player, move to the player, and then randomly choose an attack to use against the player.

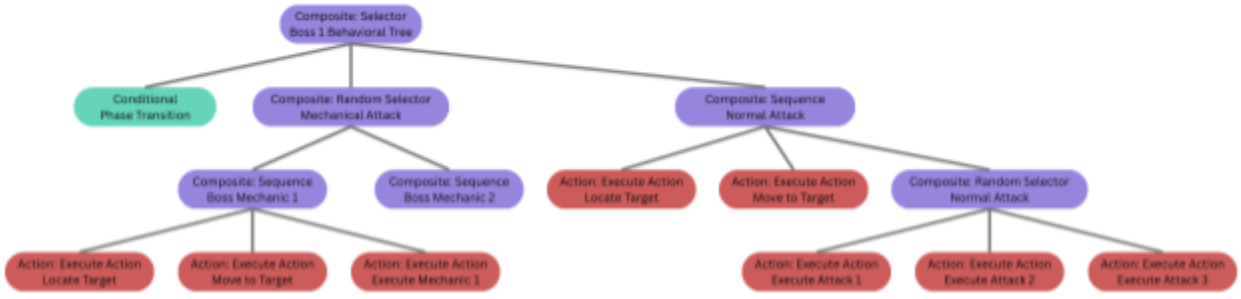

Figure 1: First Behavioral Tree Outline

For a higher-quality image of Figure 1, use this OneDrive link: Figure 1 Composite Selector.

Pathfinding Algorithms

One of the goals for both the basic and boss enemies that has gone unmentioned this far is the enemies being able to calculate a path from their current position to any given target. For the enemies to do this effectively, they will need to have access to pathfinding algorithms that best suit their design goals: this could be construed with the basic melee enemy having an “aggressive” (as mentioned in the Melee Enemy section, they will quickly close the distance and put pressure on the player) pathfinding algorithm that finds the quickest path from them to their target, or the basic ranged enemy having a pathfinding algorithm that finds the quickest safe route to a vantage point with line of sight of their target. No matter what the goal of the pathfinding algorithm may be, they need to be both time-efficient in how long it takes to process a route and accurate in how close the processed route is to the actual best route.

The method used to find the best path between two points in as few searches as possible is a widely used greedy search algorithm known as the “A* search”. This algorithm uses a heuristic function, and it hinges on said function being accurate enough to avoid supplying the algorithm with inaccurate estimates. This could lead to the “A* search” being more memory-intensive than the traditional breadth-first and depth-first search algorithms (Russell 93). To easily prevent this, restricting the search range for enemy pathfinding will help reduce the margin of error seen within the estimations of the heuristic function. While game optimization will not be a focus throughout this project, using algorithms for pathfinding that are commonly used in the game industry for the purpose of reducing memory usage is important for our team to learn. Due to this, the “A* search” will be implemented in the first semester, with easier-to-implement algorithms such as breadth-first and depth-first search being used initially as a substitute.

Another algorithm being considered for enemy pathfinding is called the jump-point search algorithm. This algorithm is one used primarily for 2D games that have a map consisting of a uniform cost grid. Jump-point search is considered one of the most efficient search

algorithms for 2D games due to its searching through each node on a 2D grid at most once. Because of this, it is noticeably faster than the “A* search” (Rabin).

Player Actions and Classes

To properly showcase the complexity of the enemy AI, the player will need to have access to attacks and skills that directly affect the decision-making process of the enemies. In the game *The Last of Us*, the developers designed an incendiary throwable weapon that would cause enemies to catch on fire. The fire also affected the pathfinding of the enemies, so they would now avoid the area on fire to prevent themselves from taking damage (if said enemies could see the fire). Whenever the enemies would be caught on fire, they would enter the “On-Fire” State and alter what actions the enemies would take next (Botta). For our game, we want to give the player the ability to immediately alter the finite state machines and behavioral trees of the enemies. To do this, we will be designing selectable player classes, each with a light attack, heavy attack, and a set of skills, each with a cooldown or cost to them. Due to the scope of this capstone, we may only be able to develop a single skill for each class, but this will be addressed towards the end of the first semester. At the start of the semester, we plan to implement two player classes, one focused on melee combat and the other on ranged. Both classes will have a light and heavy attack, with the primary differences between the two attacks being the damage they deal and the amount of time it takes for the attack to register after the button input. Generally, light attacks will deal less damage and be faster, and heavy attacks will deal more damage but be slower. We also plan on adding skills to each class; however, these skills will be focused around a certain state that inherits from one of the three parent states that will be developed: “CrowdControl”, “DamageOverTime”, and “Modifier”. For the Melee Class, we plan on adding the ability to send enemies into a “Stun” State, which would inherit from the “CrowdControl” State; alongside this, the Ranged Class will have access to inflict a “Bleed” State on the enemies, which will inherit from the “DamageOverTime” State. We are also

considering the Melee Class having access to a “Modifier” State called “Fear”, which will alter the enemy’s pathfinding; however, our primary goal for player classes in this first semester will be to develop a working Melee and Ranged Class, so this idea for class skills may be pushed back until the second semester.

Enemy State and Behavior GUI

With the project being centered around developing enemy AI and giving the player tools to directly interact with the AI logic, many bugs will probably arise with enemies falsely entering a state or being unable to enter one. To better monitor how specific state-to-state transitions occur, a GUI element will be implemented that will allow the user to select an enemy and force a switch to a specific state. The user will be able to see the current state of the enemy AI while the game is still running, as in which states the enemy is currently in, however, the option to change which state the enemy is using will require the game to be paused, this will allow the user to click on an enemy, and then choose which of their available states they want the enemy to enter from a drop-down list. This is being implemented not for the players of the game, but rather for the developers and quality assurance testers, for quicker bug fixing and user testing. However, this feature will still be available for the players to use if they want, but it is not intended to be a part of the game’s core experience. Future updates to this feature will include visualizations of the finite state machines, along with the ability to use this on boss enemies. The initial plan of having a drop-down list containing all of an enemy's states may not translate well to the bosses in the game, since each boss may have multiple instances of a behavior throughout their behavioral tree, such as the action for locating the player. As this will be much more difficult and time-consuming, a GUI element for bosses won’t be implemented until after it is implemented for the basic enemies in the game, as we will then be able to use the GUI elements from the basic enemies as the foundation on which to build the boss's GUI element.

Game Engines and Programming Languages

After careful consideration, Godot will be the game engine that will be used for this project. Specifically, Godot version 4.5. Team members tested Godot, Unity, and Unreal Engine 5 (UE5). The result of testing and research found:

Unity:

- Pros:
    - Utilizes C#, a language used for developing many famous games, such as Cuphead, Pokémon Go, and Hollow Knight.
    - Has an extensive asset store.
    - Has a large and active community, with many experts.
- Cons:
    - Unity is explicitly designed for multi-platform development, as we will only be using one platform, the main design features of Unity will be unused.
    - Combined with the fact that the layout of Unity was confusing, Unity was not a high choice for team members.

Unreal:

- Pros:
    - High focus on 3D game development.
    - The Unreal Engine's main selling point is its high-quality graphics.
    - Utilizes C++, which has been the industry standard for game creation for many years (Mooc).
    - It is very useful for game optimization and implementing a physics engine.
- Cons:
    - As it is mainly focused on 3D development, and our game will be 2D, we have no use for many of their tools.
    - We will also not be focusing on having very high-quality graphics; instead, we will

focus on a minimum of 32-bit to a maximum of 64-bit graphics.

- We do not need heavy game optimization, as we will not be making a very large game that is resource-intensive.

Godot:

- Pros:
  - Supports a wide variety of languages and has tools available to implement each of them.
  - Has a dedicated 2D rendering engine, perfect for our 2D game.
  - It is known to be far more user-friendly than Unreal or Unity.
  - This engine is designed to be lightweight, so it has fast project startup, execution, and export times, which is perfect for us and our limited time frame of one semester.
- Cons:
  - Has a limited asset and plugin ecosystem.
  - Users mention issues with its file management and cause potential crashes when attempting to move these files (Emmettwager).

The team decided Godot would be better utilized for this capstone. This is because Godot provides countless 2D capabilities, such as a tile editor and sprite editor (Introduction to 2D). It also has lightweight development tools, such as plugins and editors, that provide just what you need without slowing down the system. This is also evident by Godot being less complex and not including unnecessary 3D features that would not be used by team members.

Deciding upon the language was also an important decision for this capstone, as it needed to be a language that could handle the finite state machines, behavioral trees, and 2D development. For this capstone, GDScript, Python, and Lua were considered to be top contenders. The team members have decided that GDScript will be the initial language chosen, with Python as a secondary. Below are the reasons for this choice:

Python:

- Pros:
    - Familiarity will allow for immediate progress, as well as minimize issues because of unfamiliar syntax and or language rules.
    - Decent performance.
    - Suited for 2D games, such as EVE Online, Darkest Dungeon, or Disney's Toontown Online.
- Cons:
    - As it is an interpreted language, it is slower to execute than compiled languages such as C++ or Java.
    - Hard to integrate for Godot 4.
    - Not as fast as GDScript.
    - Limited tutorials for implementing game features.

GDScript:

- Pros:
    - Godot uses GDScript, so it is the most compatible with the engine.
    - Performance is decent, with no additional requirements for translation into GDScript.
    - Best integration with the engine and scene tree will be useful when designing multiple AI finite state machines and behavioral trees.
    - Plenty of tutorials for implementing game features.
- Cons:
    - Lower performance than compiled languages such as C#.
    - As it is tightly coupled with Godot, it limits its standalone use and portability.
    - Lacks some advanced programming features found in other languages.

    - As we have never used the language, we would have to take the time to learn the language.

Lua:

- Pros:
    - Fast (faster than Python), best used for lightweight scripting.
    - Simple syntax (similar to Python).
    - Widely used in game scripting (such as the AI in WoW).
- Cons:
    - Has limited error handling and debugging tools
    - Has a limited standard library and features.
    - As with GDScript, we have never used this language, and we will also need to take time to learn the language.

As a final decision, GDScript will be used primarily. Python will be used as a secondary language that will be used for portions of the code that GDScript may not excel at. This means Python will only be used on an as-needed basis. The compatibility between GDScript and Python is sufficient to provide a large enough incentive to use whichever is best for the task at hand. Despite this, GDScript will still be used primarily, and Python may never be deemed necessary. This decision is meant to be made based on the available resources and information that team members can obtain for a specific feature.

Art Assets

Assets will be acquired through reliable websites, such as Itch.io. Free assets will be prioritized, but paid assets may be considered. These assets will be chosen with the environment, moveset, and other interactions in mind. Our goal will be to have a coherent theme across all the assets, such as high fantasy, but this may not be possible. Alongside this, some sprite models may not have animations for certain states we wish to implement, such as

being stunned, burning, or frozen: in this scenario, we may choose to recolor the base sprite to indicate a state change or have an animation effect around it. The purpose of this is to reduce the amount of time our team will spend searching for art assets and to instead make small changes to the model, such as recoloring them, to focus on the primary goals of the project. During the first few sprints of the first semester, we will be acquiring the majority of the art assets we will be using for the first two player classes, basic enemy models, the first boss model, and the assets pack for the first map. Our goal will be to find an artist who has multiple asset packs for us to use for future maps in the second semester.

Player Progression

Player Progression is something that will be implemented in the second semester, whilst the foundations for this progression will be created in the first semester. These foundations play a major part in the design of our maps and dungeons. The player character will have stats. Stats in video games refer to the amount of health and speed the player has or the amount of damage they deal. These stats will be decided in the next semester, but will be the primary way for the player to acquire a power advantage over the enemies in the game, aside from learning the behavior of the enemy AI.

For gameplay sustainability, enjoyment, and player progression, we decided on the use of checkpoints, which are rooms that are void of enemies and can sometimes be referred to as safe rooms. These checkpoints will heal the player back to full health. They will also act as a respawn point; if the player dies before hitting the next checkpoint, they start again, or respawn at the previous checkpoint.

As for the player progression model, it will be focused on gaining a resource called experience, which will increase as the player defeats basic enemies and bosses. This experience can be used to upgrade the stats of the player character. Some systems for

upgrading these stats include a menu option that can be accessed at any time, a pop-up menu that will be showcased once the player gains a specified amount of experience, or an option to upgrade the player's stats while at a checkpoint location on a dungeon map. No matter which option is chosen, GUI elements will need to be made to allow the player to increase their character's stats, along with showcasing the amount of experience the player has earned.

Map Design

Map Design is a very important design feature that needs to be considered carefully as it directly affects several parts of the game, like how AI is used, the amount of enemies, player classes, etc.. The AI will be affected because, for example, depending on the size of the map or rooms, there is a need to decide how far the AI character might roam in these rooms or across the map. This is because the map design will follow the structure of a larger map, but with concentrated areas of enemy spawns. These can be rooms, or clearings, or whatever best fits the map. Depending on the area, deciding how many enemies populate a room or across the map in general is crucial. Giving players a challenge while not overwhelming them is a delicate balance that must be found through thorough testing. If the design focus was smaller rooms, melee enemies may be prioritized. Or for larger rooms, the focus would need to be on ranged enemies or actions that can cross greater distances to get close to enemies. Should time become limited as the project progresses, our contingency plan is to prioritize functionality over detail in our maps. Alternatively, we may employ specialized tools to expedite the design process, such as AI generators or a tool called "TileMap".

There will also be several design features that, during the first semester, will have no effect except for looks, but these features will later be used to add to the environment. Some features may include:

- Objects that can block projectiles, and the player can hide behind.

- Traps that can damage the player or enemy NPCs if stepped on.
- Pots that the player can destroy, releasing environmental features, such as fire, acid, ice, or other substances/states.
- Running water can affect the player, apply a “DamageOverTime” State, or slow the player or enemy NPCs' movement.
- Traps that shoot projectiles in a set direction, either if the player or NPC walks in front of them, or a tripwire that needs to be activated.

However, none of these have been decided upon yet and are subject to change or removal, but we hope to lay the groundwork for them so we can design and perfect them next semester.

As mentioned in the Player Progression section, we plan on creating checkpoints, which will be small enemy-free rooms. The player will have to pass through these rooms before moving into the next section of the dungeon, so they will be funneled into these rooms. We will also design boss rooms. These will be rooms at the end of the dungeon that have to be beaten to finish the dungeon and move onto the next map. These rooms will be larger than the other rooms, allowing for more space when dueling the boss. After defeating the boss, several portals will appear around the boss's room; each portal will lead to a different dungeon. This allows the player to choose which dungeon they wish to try and tackle next. The closest portal will be the intended second dungeon, as we will design the dungeons with a path in mind (as in an order for the dungeons, which one we wish to be played second, third, and so forth), but the player doesn’t have to follow this path and can go in any order they choose. There will be a checkpoint placed before the boss room so that the player can respawn before the fight and try again without having to go through the entire dungeon again. The first dungeon we create will be handled like a tutorial map for our game, so the difficulty compared to the maps we create in the next semester will be much easier, as there will be fewer enemies, fewer enemy types, an easier boss, and fewer design functions/obstacles. This dungeon is solely as a beginner level for the player to get used to the game, how enemies function, and how to deal with the bosses.

Testing

Of course, for any major project, testing is something that greatly affects the overall result of the finished product. We plan on testing our game in many different ways, namely Q/A Testing (or User Testing), Unit Testing, and finally Regression Testing.

Q/A Testing

Due to how a video game functions and is used, it's simply impossible for developers to test every way a player might interact with their game. The way many major companies deal with this is through allowing people early access to their game, through things like having Q/A testers on payroll, Betas (a usually free early access version of the game), or Alphas. We plan on doing something similar by asking as many people as we can to beta-test our game. We will be asking several Computer Science majors and people not in the Computer Science major to get more rounded feedback. We will then collect our beta testers' feedback on things such as bugs, things they enjoy (gameplay mechanics, enemies), things they don’t enjoy, or maybe find something they view as weak (needs improvement). Video Games are very complex, where players can make several different choices leading to many different results, so it is impossible to test every choice and every result, so some bugs might fall under the rug.

Unit Testing

Before we start any work on a class, we will first create unit tests to test the functionality of the class and how we wish for it to work. After we have created said class, we will test it with our unit test.

Regression/Integration Testing

Before adding any new classes or other larger modifications to our code, we will first test our original code to ensure it functions, then after adding the newer code, we will rerun our tests to ensure nothing was broken or negatively affected our project.

Capstone Schedule

This schedule provides an outline of the tentative sprint schedule. During the first two sprints, familiarizing with Godot and brainstorming implementation are the priorities. This will prevent needless mistakes or changes further along the project. The next two sprints will be focused on providing a deliverable map, user and enemy interaction, and a basic first boss to expand upon. During the next two sprints, the base user class will be finished and will have the ability to fully interact with the environment and enemies. The last sprint will focus on maximizing performance and interactions between the enemy, environment, and any other factors.

Sprint 1: September 11th	● Decide on the Language to use. ● Get familiarized with Godot. ● Download Python integration for Godot. ● Set up Trello and MoSCoW analysis ● Finish the outline of the UML diagrams for the enemy and player class implementations. ● Have a concrete concept for the first player class. ● Have a concrete concept for the first basic enemy. ● Acquire assets for the first player class. ● Acquire assets for the first basic enemy. ● Brainstorm initial ideas for the first boss; this is to ensure the first boss fits the theme of the first dungeon and doesn't feel out of place. ● Brainstorm ideas for future enemies. This also means thinking about the dungeons they will belong to. This ensures that any enemy designs remain loosely coupled and can have additions implemented with ease.

Sprint 2: September 25th	● Asset collection. ● Groundwork for the first map will begin, including an asset pack and listed locations where environmental factors can be utilized by enemy and player alike. Environmental features will not be implemented this semester, but planning is crucial. Enemy locations and checkpoints will also be discussed, but not implemented. ● Initial work on the first player class and the first basic enemy will begin. This includes asset collection. ● Diagrams to outline the first base enemy's finite state machine and or behavioral tree will be created to guide development. ● Diagrams that outline the base player class will be completed to guide development. ● Considerable research on the connection between the first enemy and the first finite state machine to ensure minimal mistakes.
Sprint 3: October 9th	● Interaction between the player and base enemy is possible, such as pathfinding, and the enemy acts accordingly. This also includes any attacks or actions used by the player. The enemy is still unable to attack the player. ● Diagrams that outline the base enemy and user class implementation will be finalized. This will ensure that any future enemies/classes can be created with minimal changes to existing code. This also allows for the chance to review implemented code and its effectiveness. ● The finite state machine and behavioral tree will continue to be worked on. This is to ensure effective pathfinding and features are implemented.
Sprint 4: October 23rd	● Development of the first boss will start. Asset collection will begin for the first boss as well. ● The base user class and enemy are completed, exhibiting no obvious bugs or issues. They act as expected and feel complete. ● Research new enemy types to add to the game for variety. This will include assets and other required research. This will be used for the second basic enemy that is implemented. ● The first basic enemy is able to interact with the player via actions.

	● Second basic enemy development starts. This includes assets.
Sprint 5: November 6th	● The first boss is functional, with a basic behavioral tree. ● The first boss is fleshed out, including various interactions with the user, as well as intuitive ‘reasoning’ provided by a behavioral tree. ● The second enemy can interact with the player, such as attacking and pathfinding.
Sprint 6: November 20th	● The first map will be finished, and we will begin thorough testing. This is to ensure there are no holes, gaps, or other unintended features. This will also help with pathfinding for enemy AI. ● Extensive testing to ensure the finite state machines/behavioral trees interact with the player adequately without seeming ‘unintelligent’. ● Maintenance on the previous code to ensure efficiency. This will include finite state machines, behavioral trees, classes, and enemies. ● Refactoring existing code to ensure loose-coupled programming values was used.
Sprint 7: December 4th	● Oral presentation/interview preparation, which includes finishing what has not been done, fixing any known bugs, implementing QoL features, and other things that need to be done. ● Mock-up of the Abstract FSM compared to the Enum FSM

Course Mapping

This project will draw on the knowledge we have acquired from several different courses during our time at WCU. Most importantly, the basic coding logic, which we learned in courses CS 150/151, will be the foundation for the majority of this project and will be reflected in our code implementation. This project will require the use of several algorithms and behavioral trees, which we learned during our time in Data Structures and Algorithms. Due to the scale of this project, such as the multiple NPCs, player classes, and GUI elements, to name a few, will require proper organization and planning, while also using several patterns, such as the

Observer, Singleton patterns, as well as some others, which we learned during our studies in Software Engineering. For the NPC AI, we will begin using finite state machines, which we learned and utilized in our Organization of Programming Languages course. Of course, that being said, there is still a wide range of things we don't know, for example, how to use and implement a game engine, as well as developing graphics and the physics needed for an enjoyable gameplay experience. While we have had courses that taught us about trees and finite state machines, we do not know how to convert that knowledge into building functional and effective video game AI yet, which I am sure will take most of our time to research, understand, and utilize. We will also need to learn how to save and load the player's progress, as well as any settings they may have changed, such as their key bindings. We will also need to learn how to add audio to our video game.

Conclusion

With Wayfarer's Crawl, our goal is to create a game that has fun gameplay with engaging, in-depth AI. We want to deliver a challenging yet enjoyable experience for players, with the AI for the NPCs and bosses as the central focus of this project. By the end of this project, we aim to have fully functional, well-designed AI systems. Creating this game is something we are all very passionate about, and we believe this journey will not only be enjoyable but also an amazing opportunity to put what we have learned to the test and expand on the knowledge we have gained over our time here at WCU.

Works Cited

"Ai (FSM, Behavior Tree, Goap, Utility Ai)." Nez Framework Documentation, anshuman-kumar.gitbook.io/nez-doc/ai-fsm-behavior-tree-goap-utility-ai. Accessed 24 Sept. 2025.

Botta, Mark. Infect AI in *The Last of Us*. In *Game AI pro 2 : Collected Wisdom of Game AI Professionals*. Crc Press, 2015.

Emmettwager, et al. "Is Godot All Good?" Godot Forum, 8 June 2024, forum.godotengine.org/t/is-godot-all-good/65718.

"Introduction to 2D." Godot Engine Documentation, docs.godotengine.org/en/stable/tutorials/2d/introduction_to_2d.html. Accessed 24 Sept. 2025.

"Minecraft Dungeons." Minecraft.Net, Microsoft, www.minecraft.net/en-us/about-dungeons. Accessed 10 Sept. 2025.

Mooc. "Best Programming Languages for Game Development." MOOC.Org, MOOC.org, 13 Dec. 2021, www.mooc.org/blog/best-programming-languages-for-game-development.

Mrhappy1227. "FFXIV - Jeuno: The First Walk Alliance Raid Guide." YouTube, youtu.be/1Khxt8a4sg8?si=PVpntoMB46n0X7bY&t=65. Accessed 25 Sept. 2025.

Rabin, Steve, and Fernando Silva. JPS+: An Extreme A* Speed Optimization for Static Uniform Cost Grids. In *Game AI pro 2 : Collected Wisdom of Game AI Professionals*. Crc Press, 2015.

Russell, Stuart, and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 3rd ed., Pearson, 2010.

Yannakakis, Georgios N., et al. *Artificial Intelligence and Games*. Cham, Switzerland, Springer, 2018.